In [ ]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import confusion_matrix, classification_report, accuracy_

import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
```

In [ ]:
```
df = pd.read_csv("/content/Student.csv")

df.head()
```

Out[ ]:

	student_id	age	gender	academic_level	study_hours	self_study_hours	onli
0	1	18	Other	High School	7.64	1.56	
1	2	18	Other	High School	2.21	2.22	
2	3	22	Male	High School	3.45	0.00	
3	4	17	Other	High School	5.75	2.08	
4	5	19	Other	High School	6.83	1.72	

5 rows × 21 columns

In [ ]:
```
df.shape
df.info()
df.describe()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5000 entries, 0 to 4999
Data columns (total 21 columns):
 #   Column                Non-Null Count  Dtype
---  ------                --------------  -----
 0   student_id            5000 non-null   int64
 1   age                   5000 non-null   int64
 2   gender                5000 non-null   object
 3   academic_level        5000 non-null   object
 4   study_hours           5000 non-null   float64
 5   self_study_hours      5000 non-null   float64
 6   online_classes_hours  5000 non-null   float64
 7   social_media_hours    5000 non-null   float64
 8   gaming_hours          5000 non-null   float64
 9   sleep_hours           5000 non-null   float64
 10  screen_time_hours     5000 non-null   float64
 11  exercise_minutes      5000 non-null   int64
 12  caffeine_intake_mg    5000 non-null   int64
 13  part_time_job         5000 non-null   int64
 14  upcoming_deadline     5000 non-null   int64
 15  internet_quality      5000 non-null   object
 16  mental_health_score   5000 non-null   int64
 17  focus_index           5000 non-null   float64
 18  burnout_level         5000 non-null   float64
 19  productivity_score    5000 non-null   float64
 20  exam_score            5000 non-null   float64
dtypes: float64(11), int64(7), object(3)
memory usage: 820.4+ KB
```

Out[ ]:

	student_id	age	study_hours	self_study_hours	online_classes_h
count	5000.000000	5000.000000	5000.000000	5000.000000	5000.00
mean	2500.500000	20.520400	4.539594	2.478734	2.01
std	1443.520003	2.870406	1.821665	1.177990	0.98
min	1.000000	16.000000	0.000000	0.000000	0.00
25%	1250.750000	18.000000	3.250000	1.660000	1.32
50%	2500.500000	20.000000	4.530000	2.480000	2.01
75%	3750.250000	23.000000	5.760000	3.290000	2.69
max	5000.000000	25.000000	11.840000	7.410000	6.00

In [ ]:

```
df.isnull().sum()
```

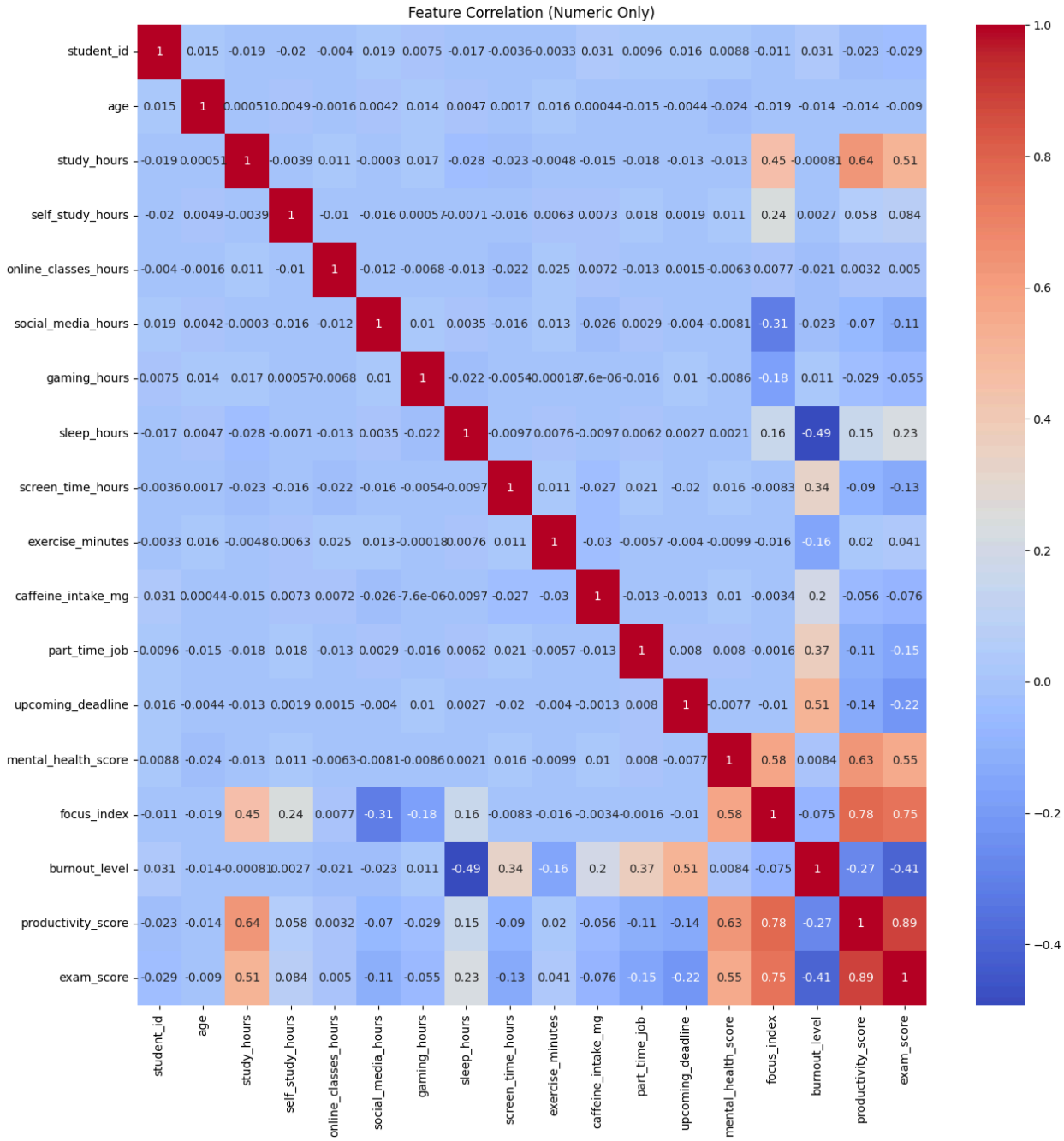
Out[ ]:

	0
student_id	0
age	0
gender	0
academic_level	0
study_hours	0
self_study_hours	0
online_classes_hours	0
social_media_hours	0
gaming_hours	0
sleep_hours	0
screen_time_hours	0
exercise_minutes	0
caffeine_intake_mg	0
part_time_job	0
upcoming_deadline	0
internet_quality	0
mental_health_score	0
focus_index	0
burnout_level	0
productivity_score	0
exam_score	0

dtype: int64

In [ ]:

```
plt.figure(figsize=(15,15))

numeric_df = df.select_dtypes(include=['int64','float64'])

sns.heatmap(numeric_df.corr(), annot=True, cmap="coolwarm")

plt.title("Feature Correlation (Numeric Only)")
plt.show()
```



In [ ]:
```
df = df.drop("student_id", axis=1)
```

In [ ]:
```
X = df.drop("exam_score", axis=1)
y = df["exam_score"]
```

In [ ]:
```
from sklearn.preprocessing import LabelEncoder

le = LabelEncoder()

categorical_cols = [
    'gender',
    'academic_level',
```

```
    'part_time_job',
    'upcoming_deadline',
    'internet_quality'
]

for col in categorical_cols:
    X[col] = le.fit_transform(X[col])
```

In [ ]:
```
X.dtypes
```

Out[ ]:

	0
age	int64
gender	int64
academic_level	int64
study_hours	float64
self_study_hours	float64
online_classes_hours	float64
social_media_hours	float64
gaming_hours	float64
sleep_hours	float64
screen_time_hours	float64
exercise_minutes	int64
caffeine_intake_mg	int64
part_time_job	int64
upcoming_deadline	int64
internet_quality	int64
mental_health_score	int64
focus_index	float64
burnout_level	float64
productivity_score	float64

dtype: object

In [ ]:
```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42
```

```
)

print("Train shape:", X_train.shape)
print("Test shape:", X_test.shape)
```

```
Train shape: (4000, 19)
Test shape: (1000, 19)
```

In [ ]:
```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test  = scaler.transform(X_test)
```

In [ ]:
```
from tensorflow import keras
from tensorflow.keras import layers

model = keras.Sequential([

    layers.Dense(128, input_shape=(X_train.shape[1],)),
    layers.BatchNormalization(),
    layers.ReLU(),

    layers.Dense(64),
    layers.BatchNormalization(),
    layers.ReLU(),

    layers.Dense(32),
    layers.BatchNormalization(),
    layers.ReLU(),

    layers.Dense(1, activation='linear')
])
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:93: User
Warning: Do not pass an `input_shape`/`input_dim` argument to a layer. When usi
ng Sequential models, prefer using an `Input(shape)` object as the first layer
in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

In [ ]:
```
model.compile(
    optimizer='adam',
    loss='mse',
    metrics=['mae']
)
```

In [ ]:
```
history = model.fit(
    X_train,
    y_train,
    epochs=50,
    batch_size=32,
    validation_split=0.2
```

```
)
```

```
Epoch 1/50
100/100 ━━━━━━━━━━━━━━━━━━━━ 3s 5ms/step - loss: 500.0320 - mae: 19.5330 - va
l_loss: 395.2590 - val_mae: 17.7172
Epoch 2/50
100/100 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 385.4306 - mae: 17.3556 - va
l_loss: 323.1823 - val_mae: 16.1205
Epoch 3/50
100/100 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 286.3426 - mae: 14.9761 - va
l_loss: 214.8848 - val_mae: 13.0812
Epoch 4/50
100/100 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 204.9904 - mae: 12.4051 - va
l_loss: 148.4844 - val_mae: 10.6209
Epoch 5/50
100/100 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 124.5868 - mae: 9.3796 - val_l
oss: 78.9483 - val_mae: 7.2702
Epoch 6/50
100/100 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 62.2424 - mae: 6.3464 - val_lo
ss: 55.7953 - val_mae: 5.9479
Epoch 7/50
100/100 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 44.5058 - mae: 5.2076 - val_lo
ss: 36.6602 - val_mae: 4.7601
Epoch 8/50
100/100 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 35.8105 - mae: 4.6104 - val_lo
ss: 31.0617 - val_mae: 4.3862
Epoch 9/50
100/100 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 27.9866 - mae: 4.1607 - val_lo
ss: 25.6643 - val_mae: 4.0341
Epoch 10/50
100/100 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 26.4759 - mae: 4.0474 - val_lo
ss: 24.4192 - val_mae: 3.8936
Epoch 11/50
100/100 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 26.0599 - mae: 4.0116 - val_lo
ss: 25.3407 - val_mae: 4.0136
Epoch 12/50
100/100 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 25.0314 - mae: 3.9257 - val_lo
ss: 25.3079 - val_mae: 3.9808
Epoch 13/50
100/100 ━━━━━━━━━━━━━━━━━━━━ 1s 6ms/step - loss: 26.0475 - mae: 3.9773 - val_lo
ss: 25.7855 - val_mae: 4.0391
Epoch 14/50
100/100 ━━━━━━━━━━━━━━━━━━━━ 1s 6ms/step - loss: 23.0284 - mae: 3.7609 - val_lo
ss: 26.7728 - val_mae: 4.1643
Epoch 15/50
100/100 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 26.0523 - mae: 4.0084 - val_lo
ss: 28.5375 - val_mae: 4.2494
Epoch 16/50
100/100 ━━━━━━━━━━━━━━━━━━━━ 1s 6ms/step - loss: 23.9790 - mae: 3.8103 - val_lo
ss: 26.7053 - val_mae: 4.0825
Epoch 17/50
100/100 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 23.1412 - mae: 3.7331 - val_lo
ss: 27.3561 - val_mae: 4.1228
Epoch 18/50
100/100 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 21.8775 - mae: 3.6865 - val_lo
ss: 26.7473 - val_mae: 4.0859
```

```
Epoch 19/50
100/100 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 24.5054 - mae: 3.8960 - val_lo
ss: 26.3099 - val_mae: 4.0244
Epoch 20/50
100/100 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 23.4358 - mae: 3.7808 - val_lo
ss: 26.4200 - val_mae: 4.0813
Epoch 21/50
100/100 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 19.8103 - mae: 3.4627 - val_lo
ss: 31.8328 - val_mae: 4.4633
Epoch 22/50
100/100 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 21.6484 - mae: 3.6365 - val_lo
ss: 28.7736 - val_mae: 4.2513
Epoch 23/50
100/100 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 23.1104 - mae: 3.7562 - val_lo
ss: 30.4203 - val_mae: 4.4065
Epoch 24/50
100/100 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 20.1576 - mae: 3.5237 - val_lo
ss: 28.1074 - val_mae: 4.1764
Epoch 25/50
100/100 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 20.2013 - mae: 3.5294 - val_lo
ss: 29.3888 - val_mae: 4.2477
Epoch 26/50
100/100 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 19.9085 - mae: 3.5145 - val_lo
ss: 29.2229 - val_mae: 4.2518
Epoch 27/50
100/100 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 19.9755 - mae: 3.4981 - val_lo
ss: 28.2003 - val_mae: 4.2074
Epoch 28/50
100/100 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 21.9871 - mae: 3.6659 - val_lo
ss: 27.7196 - val_mae: 4.1868
Epoch 29/50
100/100 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 17.6927 - mae: 3.2992 - val_lo
ss: 29.0640 - val_mae: 4.2695
Epoch 30/50
100/100 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 19.8853 - mae: 3.4957 - val_lo
ss: 28.8001 - val_mae: 4.1724
Epoch 31/50
100/100 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 20.5228 - mae: 3.5721 - val_lo
ss: 28.5401 - val_mae: 4.2059
Epoch 32/50
100/100 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 17.0985 - mae: 3.2519 - val_lo
ss: 29.2094 - val_mae: 4.2795
Epoch 33/50
100/100 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 17.2514 - mae: 3.2504 - val_lo
ss: 29.7902 - val_mae: 4.3007
Epoch 34/50
100/100 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 16.1023 - mae: 3.1134 - val_lo
ss: 29.7793 - val_mae: 4.2516
Epoch 35/50
100/100 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 17.4886 - mae: 3.2745 - val_lo
ss: 29.9451 - val_mae: 4.3279
Epoch 36/50
100/100 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 19.6528 - mae: 3.4408 - val_lo
ss: 32.5054 - val_mae: 4.4698
```

```
Epoch 37/50
100/100 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 18.9211 - mae: 3.3967 - val_lo
ss: 30.4143 - val_mae: 4.3351
Epoch 38/50
100/100 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 15.2621 - mae: 3.0345 - val_lo
ss: 29.6819 - val_mae: 4.2982
Epoch 39/50
100/100 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 17.2859 - mae: 3.2538 - val_lo
ss: 30.1155 - val_mae: 4.3414
Epoch 40/50
100/100 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 17.4450 - mae: 3.2181 - val_lo
ss: 30.1614 - val_mae: 4.2965
Epoch 41/50
100/100 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 17.3952 - mae: 3.2762 - val_lo
ss: 30.7456 - val_mae: 4.3016
Epoch 42/50
100/100 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 14.8260 - mae: 3.0245 - val_lo
ss: 31.7358 - val_mae: 4.4380
Epoch 43/50
100/100 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 17.8698 - mae: 3.2842 - val_lo
ss: 32.0927 - val_mae: 4.4508
Epoch 44/50
100/100 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 17.4940 - mae: 3.2169 - val_lo
ss: 30.1711 - val_mae: 4.2784
Epoch 45/50
100/100 ━━━━━━━━━━━━━━━━━━━━ 1s 6ms/step - loss: 16.2708 - mae: 3.1218 - val_lo
ss: 32.2231 - val_mae: 4.4537
Epoch 46/50
100/100 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - loss: 17.7342 - mae: 3.2964 - val_lo
ss: 30.4011 - val_mae: 4.2981
Epoch 47/50
100/100 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - loss: 16.7940 - mae: 3.2076 - val_lo
ss: 31.0565 - val_mae: 4.3355
Epoch 48/50
100/100 ━━━━━━━━━━━━━━━━━━━━ 1s 6ms/step - loss: 14.7715 - mae: 3.0168 - val_lo
ss: 31.4539 - val_mae: 4.3659
Epoch 49/50
100/100 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 16.8438 - mae: 3.1597 - val_lo
ss: 31.0677 - val_mae: 4.3664
Epoch 50/50
100/100 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 15.7772 - mae: 3.1082 - val_lo
ss: 33.1692 - val_mae: 4.4427
```

In [ ]:
```
loss, mae = model.evaluate(X_test, y_test)

print("Test MAE:", mae)
```

```
32/32 ━━━━━━━━━━━━━━━━━━━━ 1s 9ms/step - loss: 33.8184 - mae: 4.5444
Test MAE: 4.534809112548828
```

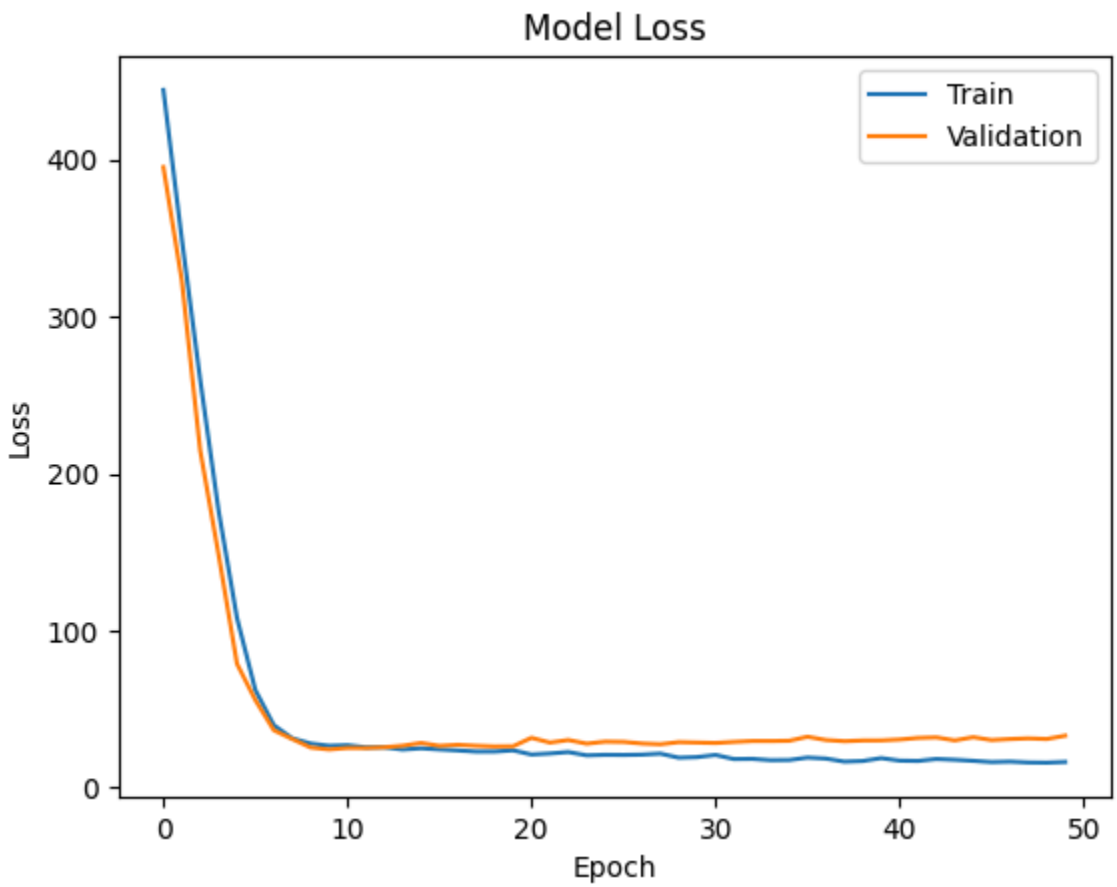
In [ ]:
```
y_pred = model.predict(X_test)
```

```
32/32 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step
```

In [ ]:
```
from sklearn.metrics import mean_squared_error, r2_score

mse = mean_squared_error(y_test, y_pred)
r2  = r2_score(y_test, y_pred)

print("MSE:", mse)
print("R2 Score:", r2)
```

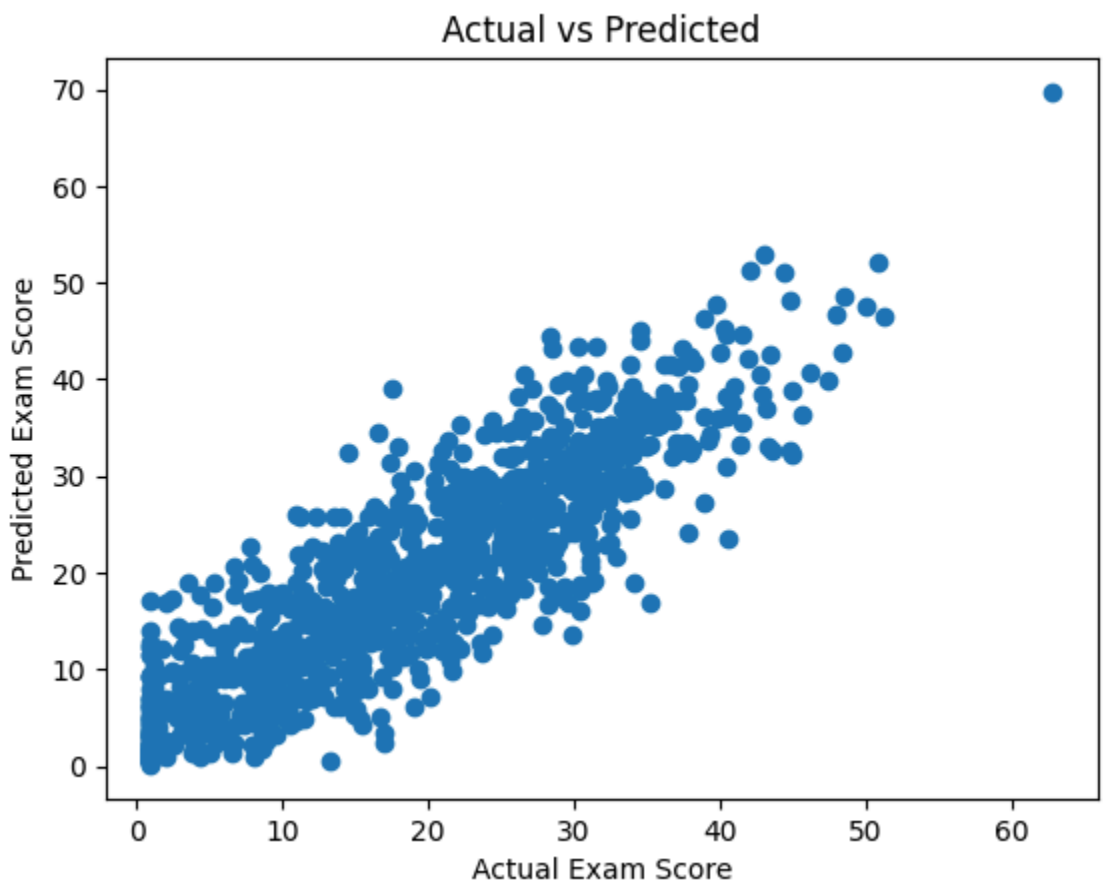
```
MSE: 34.123556943542035
R2 Score: 0.7495702741688751
```

In [ ]:
```
import matplotlib.pyplot as plt

plt.plot(history.history['loss'])
plt.plot(history.history['val_loss'])

plt.title("Model Loss")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.legend(["Train","Validation"])
plt.show()
```



In [ ]:
```
plt.scatter(y_test, y_pred)
plt.xlabel("Actual Exam Score")
plt.ylabel("Predicted Exam Score")
plt.title("Actual vs Predicted")
```

```
plt.show()
```


Actual vs Predicted
Predicted Exam Score
Actual Exam Score
0
10
20
30
40
50
60
70